

Esercizio 1

Data la seguente porzione di programma rispondere alle domande corrispondenti **motivando tutte le risposte date**:

```
int main() {  
  
    int* matricola = new int[6]{...inserisci la tua matricola...};  
    int* p = new int[3];  
  
    // 1. Le seguenti istruzioni sono corrette?  
    for (int i=0; i < 3; i++)  
        p[i] = matricola[i];  
  
    // 2. Le seguenti istruzioni sono corrette? Se sì, che cosa stampano?  
    if (&(p[0]) == &(matricola[0])) cout << "Yes" << endl;  
    else cout << "No" << endl;  
  
    // 3. Le seguenti istruzioni sono corrette? se sì, che cosa stampano?  
    for (int i=0; i<3; ++i) {  
        *(p+i) += matricola[i+3];  
        cout << p[i] << " - " << matricola[i] << endl;  
    }  
  
    // 4. Scrivere sul foglio le operazioni necessarie per deallocare la  
    memoria dinamica utilizzata  
  
    return 0;  
  
}
```

Esercizio 2

Si consideri un Pronto Soccorso semplificato, in cui ad ogni paziente viene attribuito uno tra i seguenti codici di gravità: ROSSO, GIALLO, VERDE. Modellare opportunamente un paziente nella classe `Paziente`, tenendo conto che un paziente è caratterizzato da: un ID univoco che lo identifica, un codice di gravità (ROSSO, GIALLO, VERDE), la sua età.

Implementare inoltre una classe `CodaProntoSoccorso` che tenga traccia dell'ordine in cui visitare i pazienti in sala d'attesa secondo i seguenti criteri di priorità:

- I pazienti con codice ROSSO hanno priorità su quelli di codice GIALLO, che hanno priorità su quelli di codice VERDE;
- a parità di codice, viene visitato prima il paziente più anziano;
- a parità di età, viene visitato prima chi è arrivato prima, secondo l'ordine di inserimento nella coda.

La classe `CodaProntoSoccorso` deve fornire i seguenti metodi:

1. `bool arrivo(Paziente* p)`: se non esiste alcun paziente con lo stesso ID di `p` già nella sala d'attesa, inserisce il paziente `p` secondo le priorità e ritorna `true`; altrimenti, non altera la sala d'attesa e ritorna `false`;
2. `Paziente* visitaProssimo()`: restituisce il prossimo paziente, eliminandolo dalla sala d'attesa. Se quest'ultima è vuota, restituisce `nullptr`.
3. `Paziente* checkProssimo() const`: restituisce il prossimo paziente, senza eliminarlo dalla sala d'attesa. Se quest'ultima è vuota, restituisce `nullptr`;

4. la ridefinizione di `operator>`: siano `a` e `b` due istanze di `CodaProntoSoccorso`. `a>b` se il numero di pazienti con codice ROSSO di `a` > di numero di pazienti con codice ROSSO di `b`. A parità di pazienti con codice ROSSO, si confronta sui pazienti con codice GIALLO. A parità di pazienti con codice GIALLO, si confronta sui pazienti con codice VERDE.

Si può assumere che gli input siano sempre corretti, che il codice di gravità sia sempre uno tra ROSSO, GIALLO, VERDE e che il tempo di attesa sia un intero ≥ 0 .

Ai fini dell'esercizio, non è consentito l'uso di strutture dati STL ad eccezione di `vector` e `list`. Criteri che influenzeranno la valutazione dell'esercizio sono anche: la scelta delle strutture dati utilizzate per rappresentare il problema e l'efficienza dei metodi implementati.

Esercizio 3

Dato in input un albero binario `tree`, implementato tramite la classe `AlberoB<int> tree`, implementare una funzione `esercizio3(...)` che restituisca `true` se e solo se, per ogni livello, la somma dei valori di quel livello è minore della somma dei valori del livello successivo (se esiste). In caso contrario, la funzione deve restituire `false`.

Si assuma che la radice si trovi al livello 0.

La classe `AlberoB<T>` mette a disposizione la seguente interfaccia pubblica di metodi costanti (`tree` istanza di `AlberoB<T>`):

- `tree.radice()` restituisce il valore informativo di `tree` (di tipo `T`)
- `tree.figlio(DIR)` restituisce il sottoalbero sinistro (`DIR=SIN`) e destro (`DIR=DES`) di `tree`
- `tree.nullo()` restituisce `true` se `tree` è un albero nullo e `false` altrimenti
- `tree.foglia()` restituisce `true` se `tree` è una foglia e `false` altrimenti

Esercizio 4

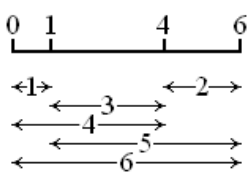
Un **regolo di Golomb** è un particolare tipo di regolo in cui il numero di tacche viene detto **ordine**, mentre la massima distanza tra due delle sue tacche viene detta **lunghezza**.

Le tacche di un regolo di Golomb rispettano i seguenti vincoli:

- non esiste alcuna coppia di tacche poste alla stessa distanza;
- come per i regoli tradizionali, le sue tacche devono essere disposte in ordine crescente;
- la prima tacca si trova in posizione 0, mentre l'ultima in posizione `l`, dove `l` è la lunghezza del regolo.

Scrivere una funzione `esercizio4` che, presi in input un `unsigned o` e un `unsigned l`, componga un `vector<unsigned> regolo` che rappresenti le tacche di un regolo di Golomb di ordine `o` e lunghezza `l`. Se tale regolo esiste, la funzione deve stampare in output il vettore `regolo`; altrimenti, deve stampare "IMPOSSIBILE".

Esempio: il seguente è un regolo di Golomb di ordine 4 e lunghezza 6. Si noti che la distanza tra ogni coppia di tacche è sempre diversa. In questo caso, `regolo = {0, 1, 4, 6}`.

	Esempio: il seguente è un regolo di Golomb di ordine 4 e lunghezza 6. Si noti che la distanza tra ogni coppia di tacche è sempre diversa. In questo caso, <code>regolo = {0, 1, 4, 6}</code> .
---	---